

Universidad Torcuato Di Tella

Métodos Analíticos y Programación

Prof. Hernán Czemerinski

Teórica 7: Clases y objetos (parte I)

Clases y objetos

Python, nativamente, admite valores de distintos tipos:

- ▶ 7, 73 (`int`)
- ▶ False, True (`bool`)
- ▶ [1,2,3,4] (`list`)
- ▶ {'BA':'Buenos Aires', 'CF':'Capital Federal'} (`dict`)

Provee operaciones para manipularlos:

- ▶ 7 + 73
- ▶ False and True
- ▶ [1,2,3,4].append(5)
- ▶ 'BA' in {'BA':'Buenos Aires', 'CF':'Capital Federal'}

Clases y objetos

Usando int, float, bool, str, list, set y dict resolvimos diferentes problemas.

Ejemplos:

```
# Suma los números de una lista de números
def suma(numeros):
    suma = 0
    for numero in numeros:
        suma = suma + numero
    return suma

# Determina si un año es bisiesto
def es_bisiesto(a):
    return (a%400 == 0) or (a%4 == 0 and a%100 != 0)

# Devuelve el conjunto de letras de una palabra
def letras(palabra):
    rv = set()
    for letra in palabra:
        rv.add(letra)
    return rv
```

Clases y objetos

Supongamos que quisiéramos escribir un programa para venta de calzados. ¿Cómo modelamos el problema?



Sería (*¡muy!*) bueno contar con variables de tipo Calzado y CarritoDeCompras para escribir el programa.

```
c1 = Calzado('Puma', 'Borussia', 8699.99)
c2 = Calzado('Adidas', 'New York', 7800.00)

carrito = CarritoDeCompras()
carrito.agregar(c1)
carrito.agregar(c2)

print('Monto total: $', carrito.monto_total())
```

La **programación orientada a objetos** (OOP) es un paradigma de programación que permite definir tipos usando **clases** para modelar problemas con **objetos** que funcionan como representaciones de entidades del mundo real.

Ej: Java, Smalltalk, C++, Python, etc.

Clases y objetos

Una **clase** es un “plano” que define las **operaciones** y la **representación interna** de los objetos del nuevo tipo.

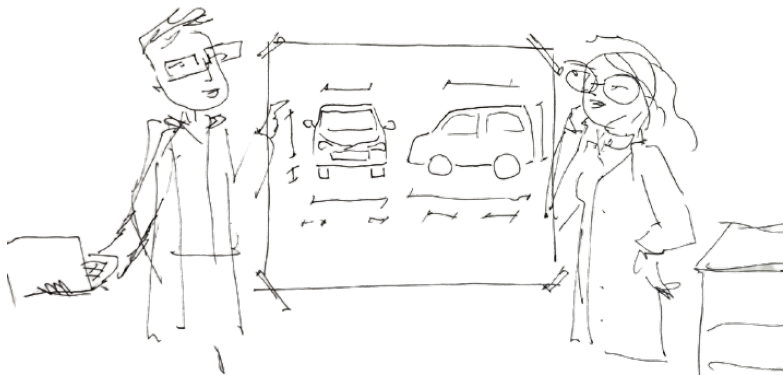


Ilustración: Klinko

Clases y objetos

Los **objetos** son **instancias** del nuevo tipo que se manipulan con las operaciones definidas en la clase.



Ilustración: Klinko

Clases: uso

Una clase provee los mecanismos necesarios para que un programa cliente pueda crear y manipular sus objetos.

Ejemplo:

- ▶ `c = Calzado(mar, mod, pre):` Crea un nuevo calzado con la marca, el modelo y el precio especificados.
- ▶ `c.marca:` marca del calzado `c`.
- ▶ `c.modelo:` modelo del calzado `c`.
- ▶ `c.precio:` precio del calzado `c`.
- ▶ `c.aplicar_descuento(porcentaje):` aplica el descuento indicado en el precio de `c`.

```
c = Calzado('Topper', 'Tenis', 10000.00)

print(c.marca)      # imprime 'Topper'
print(c.modelo)     # imprime 'Tenis'
print(c.precio)     # imprime 10000.00

c.aplicar_descuento(20)
print(c.precio)     # imprime 8000.00
```

Una clase define:

- ▶ **Atributos:** variables internas que guardan información de cada objeto (ej: marca, modelo y precio).
- ▶ **Métodos:** funciones que definen lo que el objeto puede hacer (ej: `aplicar_descuento(...)`).

Clases: uso

Una clase provee los mecanismos necesarios para que un programa cliente pueda crear y manipular sus objetos.

Cada objeto posee sus propios valores en sus atributos.

Ejemplo:

```
c1 = Calzado('Puma', 'Borussia', 8699.99)
c2 = Calzado('Adidas', 'New York', 7800.00)

print(c1.marca)      # imprime 'Puma'
print(c2.marca)      # imprime 'Adidas'

print(c1.modelo)     # imprime 'Borussia'
print(c2.modelo)     # imprime 'New York'

print(c1.precio)     # imprime 8699.99
print(c2.precio)     # imprime 7800.00
```


Clases: uso

Ejercicio 1. Se cuenta con una clase Automovil:

- ▶ `Automovil(mar, mod, vel_max)`: Crea un automóvil, con la marca, el modelo y velocidad máxima indicados.
mar y mod: str; vel_max: int.
- ▶ `a.marca`: marca del auto a.
- ▶ `a.modelo`: modelo del auto a.
- ▶ `a.velocidad_maxima`: velocidad máxima del auto a.

Implementar una función `marca_auto_mas_veloz(autos)`, que dada una lista no vacía de automóviles devuelva la marca del automóvil más veloz de la lista.

Para resolverlo, bajar el archivo `templates.zip` y completar la función en `ej1/ej1_prog_principal.py`. A continuación puede verse un ejemplo del resultado de una invocación:

```
auto1 = Automovil("Renault", "Sandero", 172)
auto2 = Automovil("Ford", "Fiesta", 190)
auto3 = Automovil("Volkswagen", "Surán", 167)
auto4 = Automovil("Fiat", "Punto", 181)

autos = [auto1, auto2, auto3, auto4]

# La siguiente línea debería imprimir "La marca del auto más veloz es Ford."
print("La marca del auto más veloz es " + marca_auto_mas_veloz(autos) + ".")
```

Clases: implementación

Hasta ahora trabajamos con valores de diferentes tipos.

```
peso_atómico = dict()
peso_atómico['H'] = 1.00797
peso_atómico['He'] = 4.0026
peso_atómico['C'] = 12.0111
peso_atómico['U'] = 238.02891

felinos = set()
felinos.add('león')
felinos.add('gato')
```

¿Cómo hace Python para representarlos internamente?

No sabemos...

Clases: implementación

```
class Calzado:

    def __init__(self, mar, mod, pre):
        self.marca = mar
        self.modelo = mod
        self.precio = pre
    ...

# Cuerpo principal del programa
c = Calzado('Converse', 'All Star', 7199.00)
print(c.marca)    # Imprime 'Converse'
...
```

- ▶ La palabra `class` indica que se está definiendo una clase; a continuación se escribe su nombre seguido del carácter `:`.
- ▶ El método `__init__` se ejecuta cada vez que se crea un objeto.
- ▶ En este caso, `self` representa al objeto que se está construyendo.
- ▶ Para referenciar a los `atributos` (variables internas) de los objetos se usa la notación `self.<propiedad>`

En el ejemplo, en las propiedades `marca`, `modelo` y `precio` del objeto `c` se almacenarán los valores `'Converse'`, `'All Star'` y `7199.00`, respectivamente.

Clases: implementación

Los métodos definen las operaciones o reglas de negocio que determinan cómo se comportan los objetos de la clase.

```
class Calzado:

    def __init__(self, mar, mod, pre):
        self.marca = mar
        self.modelo = mod
        self.precio = pre

    def aplicar_descuento(self, porcentaje):
        self.precio = self.precio * (1 - porcentaje / 100)

    ...

# Cuerpo principal del programa
c = Calzado('Topper', 'Tenis', 10000.00)
print(c.precio)           # Imprime 10000.00
c.aplicar_descuento(25)   # Modifica el precio de c
print(c.precio)           # Imprime 7500.00
...
```

En el método `aplicar_descuento(...)`, `self` representa al objeto sobre el que se invoca el método.

Clases: implementación

Ejercicio 2. Implementar la clase Cuadrado del archivo ej2/clase_cuadrado.py con los siguientes atributos y métodos:

- ▶ `c = Cuadrado(long_lado):` Crea un cuadrado con longitud de lado `long_lado`.
- ▶ `c.lado:` longitud de cada lado del cuadrado `c`.
- ▶ `c.perimetro():` Devuelve el perímetro del cuadrado `c`.
- ▶ `c.superficie():` Devuelve la superficie del cuadrado `c`.

Ejemplo de uso:

```
c1= Cuadrado(5.0)
print('Lado de c1: ' + str(c1.lado)                # 5.0
print('Per. de c1: ' + str(c1.perimetro()))        # 20.0
print('Sup. de c1: ' + str(c1.superficie()))       # 25.0

c2= Cuadrado(2.0)
print('Lado de c2: ' + str(c2.lado))               # 2.0
print('Per. de c2: ' + str(c2.perimetro()))        # 8.0
print('Sup. de c2: ' + str(c2.superficie()))       # 4.0
```

Clases: implementación

Ejercicio 3. (Tarea) En el archivo del archivo `ej3/clase_carrito_compras.py`, implementar el método `monto_total()` de la clase `CarritoDeCompras`.

- ▶ `c = CarritoDeCompras()`: Crea un carrito de compras vacío.
- ▶ `c.agregar(calzado)`: Agrega calzado al carrito `c`.
- ▶ `c.eliminar(calzado)`: Elimina una aparición de calzado del carrito `c`.
- ▶ `c.monto_total()`: Devuelve el monto correspondiente a todos los calzados del carrito `c`.

Notar que la clase `CarritoDeCompras` define un único atributo llamado `contenido`, que es una lista que permite conservar los calzados que son incorporados o eliminados del carrito.

Para probar si el comportamiento es el esperado ejecutar el programa `ej2_prog_principal.py`.

Clases: implementación

Ejercicio 3. (continuación)

Ejemplo de uso:

```
from clase_calzado import Calzado
from clase_carrito_compras import CarritoDeCompras

calzado1 = Calzado("Puma", "Borussia", 8699.99)
calzado2 = Calzado("Adidas", "New York", 7800.00)

carrito = CarritoDeCompras()
carrito.agregar(calzado1)
carrito.agregar(calzado2)
print(carrito.monto_total())           # Imprime '16499.99'

carrito.eliminar(calzado2)
print(carrito.monto_total())           # Imprime '8699.99'
```

Nota: Se usa `from <archivo> import <clase>` para usar clases definidas en otros archivos.

Repaso de la clase de hoy

- ▶ Clases y objetos
 - ▶ Definición y uso
 - ▶ Métodos
 - ▶ Propiedades

Bibliografía complementaria

- ▶ IPP, capítulos 14, 15, 16 y 18
- ▶ The Python Tutorial, sección 9

Con lo visto, ya pueden hacer hasta el ejercicio 5 de la sección 8 de la guía de ejercicios.